BIRZEIT UNIVERSITY

Task #2 - Symmetric Crypto Systems: Implementation and Analysis

Prepared by:

Tasneem Shelleh 1220439

Hala Sabobeh 1220322

Lara Abu Asfour 1221484

Date: 12/8/2025

Overview of the Implementation:

Core AES implementation:

In this part, we built a complete AES-128 encryption and decryption system entirely from scratch (no precomputed lookup tables or library shortcuts) Our implementation starts with the foundation: Galois Field arithmetic in GF(2^8) which is an essential part of AES's byte-level operations. We implemented finite field multiplication, exponentiation, and multiplicative inverse functions, along with the AES specific affine transformation, to dynamically generate both the substitution function and its inverse. This means the substitution layer (SubBytes) and its reverse (InvSubBytes) do not rely on static tables, but instead are computed according to the AES specification.

That we implemented each function from scratch:

- SubBytes / InvSubBytes: Non-linear byte substitution.
- ShiftRows / InvShiftRows: Row shift byte rotations to diffuse data across columns.
- MixColumns / InvMixColumns: Column mixing,that Galois Field mixing for further diffusion.
- AddRoundKey: XOR based combination of state bytes with round keys.

The round keys themselves are produced via our generated Key Expansion function, which uses word rotations, byte substitutions, and round constant, to generate the 11 round keys needed for AES-128 (one for the initial AddRoundKey and one for each of the 10 rounds).

With the building blocks complete, we implemented the AES block cipher for both encryption and decryption. Each process exactly one 16-byte block, applying the full AES round structure: initial AddRoundKey, nine main rounds (SubBytes → ShiftRows → MixColumns → AddRoundKey), and the final round without MixColumns.

CBC Mode Integration:

To extend the block cipher to encrypt arbitrary-length messages, we implemented Cipher Block Chaining (CBC) mode. The CBC mode uses an Initialization Vector (IV) to randomize encryption for the first block, ensuring that identical plaintexts produce different ciphertexts when different IVs are used. Each plaintext block is XORed with the previous ciphertext block before encryption, creating a dependency chain across the message and eliminating repeating block patterns. This chaining means that a change in one block affects all subsequent blocks, increasing security.

Padding: we also implemented PKCS#7 padding to handle cases where the plaintext length is not an exact multiple of the 16-byte AES block size. This method makes sure the data always fits perfectly into the required block structure. When encrypting, the program calculates how many extra bytes are needed and appends that number of bytes to the plaintext, with each added byte containing a value equal to the total number of padding bytes. When decrypting, it checks the last byte to determine how much padding was added, verifies that all the padding bytes are correct, and then removes them to recover the original message. We also built in strict validation during decryption so that any incorrect or intentionally corrupted padding is detected and rejected, helping to protect the integrity of the data.

Additionally, we developed helper functions, and block splitting/printing utilities. These tools make CBC encryption and decryption easier to test, debug, and visualize in a block-by-block format.

Sample Input/Output

After running task2_run_aes.py code, we got:

Encryption Example:

Key (hex): 2b7e151628aed2a6abf7158809cf4f3c

IV (hex): 000102030405060708090a0b0c0d0e0f

Plaintext (hex): 6bc1bee22e409f96e93d7e117393172a

Ciphertext (hex) (2 Blocks): [01] 7649abac8119b246Cee98e9b12e9197d

[02] 8964e0b149c10b7b682e6e39aaeb731c

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\user\OneDrive\Desktop\crypto_task2> & C:\Users\user\AppData\Local\Programs\Python\Python312\python.exe c:/Users/user/OneDrive/Desktop/crypto_task2/task2_run_aes.py
AES-128 CBC Runner
Encrypt (E) or Decrypt (D): E
Enter 128-bit AES key: 2b7e151628aed2a6abf7158809cf4f3c
Enter 128-bit IV: 000102030405060708090a0b0c0d0e0f
Enter plaintext: 6bc1bee22e409f96e93d7e117393172a
Ciphertext blocks (hex) (2 blocks):
  [01] 7649abac8119b246cee98e9b12e9197d
  [02] 8964e0b149c10b7b682e6e39aaeb731c

[Padding info]
  Plaintext length: 16 bytes
  Padding length: 16 byte(s)
  Padding byte value: 0x10  (bits: 00010000)
  Last ciphertext block: 8964e0b149c10b7b682e6e39aaeb731c
  Extra padding block was added (plaintext was exact multiple of 16).
  Padding bits (repeated 16 times): 00010000 00010000 00010000 00010000 00010000 00010000 00010000 00010000 00010000 00010000 00010000 00010000 00010000 00010000 00010000 00010000

Raw ciphertext (hex):
7649abac8119b246cee98e9b12e9197d8964e0b149c10b7b682e6e39aaeb731c
PS C:\Users\user\OneDrive\Desktop\crypto_task2>
```

Decryption Example:

Key (hex): 2b7e151628aed2a6abf7158809cf4f3c

IV (hex): 000102030405060708090a0b0c0d0e0f

Ciphertext (hex): 7649abac8119b246cee98e9b12e9197d8964e0b149c10b7b682e6e39aaeb731c

Recovered Plaintext (hex) (after unpadding, 1 block):[01] 6bc1bee22e409f96e93d7e117393172a

```
PS C:\Users\user\OneDrive\Desktop\crypto_task2> & C:\Users\user\AppData\Local\Programs\Python\Python312\python.exe c:/Use
rs/user/OneDrive/Desktop/crypto_task2/task2_run_aes.py
AES-128 CBC Runner
Encrypt (E) or Decrypt (D): D
Enter 128-bit AES key: 2b7e151628aed2a6abf7158809cf4f3c
Enter 128-bit IV: 000102030405060708090a0b0c0d0e0f
Enter ciphertext : 7649abac8119b246cee98e9b12e9197d8964e0b149c10b7b682e6e39aaeb731c
Plaintext blocks (hex, after unpadding) (1 block):
  [01] 6bc1bee22e409f96e93d7e117393172a

Raw plaintext (hex):
6bc1bee22e409f96e93d7e117393172a
PS C:\Users\user\OneDrive\Desktop\crypto_task2>
```

In the example above, the plaintext was encrypted using AES-128 in CBC mode with the given key and IV, producing the ciphertext shown; when the ciphertext was decrypted with the same key and IV, the original plaintext was fully recovered, confirming that the encryption–decryption process works correctly.

Avalanche Effect Results & Interpretation

We run task2_aes_avalanche_analysis.py to perform 10 trials each for:

A. Flipping a single bit in plaintext
B. Flipping a single bit in the key

Results of Plaintext Bit Flip[A]:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\user\OneDrive\Desktop\crypto_task2> & C:\Users\user\AppData\Local\Programs\Python\Python312\pyth
rs/user/OneDrive/Desktop/crypto_task2/task2_aes_avalanche_analysis.py
Avalanche Effect
[A] Flip 1 bit in Plaintext (10 rounds)
  Trial 01: flipped P bit #110, Bits different = 127 (49.61%)
  Trial 02: flipped P bit #3, Bits different = 145 (56.64%)
  Trial 03: flipped P bit #3, Bits different = 145 (56.64%)
  Trial 04: flipped P bit #20, Bits different = 111 (43.36%)
  Trial 05: flipped P bit #35, Bits different = 132 (51.56%)
  Trial 06: flipped P bit #49, Bits different = 122 (47.66%)
  Trial 07: flipped P bit #46, Bits different = 122 (47.66%)
  Trial 08: flipped P bit #4, Bits different = 116 (45.31%)
  Trial 09: flipped P bit #76, Bits different = 127 (49.61%)
  Trial 10: flipped P bit #86, Bits different = 124 (48.44%)
Average bits different (A): 127.10 / 256 (49.65%)
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Summary (A) Plaintext bit flip:
Trial | Flipped bit idx (P) | Bits different | %
------+---------------------+----------------+-------
01    | 110                 | 127            | 49.61%
02    | 3                   | 145            | 56.64%
03    | 3                   | 145            | 56.64%
04    | 20                  | 111            | 43.36%
05    | 35                  | 132            | 51.56%
06    | 49                  | 122            | 47.66%
07    | 46                  | 122            | 47.66%
08    | 4                   | 116            | 45.31%
09    | 76                  | 127            | 49.61%
10    | 86                  | 124            | 48.44%
```

Average Bits Different: 127.10 / 256 = 0.49648

Average % Different: 49.65%

In this test, we checked AES's avalanche effect by flipping just one bit in the plaintext and observing how much the ciphertext changed. Across 10 trials, the number of bits that changed in the ciphertext averaged about 127 out of 256 bits (49.65%). This is very close to the ideal 50%, showing that a tiny change in the input spreads widely through the encryption process, which is exactly what a secure cipher should do.

Results of Key Bit Flip:

```
[B] Flip 1 bit in Key (10 rounds)
  Trial 01: flipped K bit #20, Bits different = 132 (51.56%)
  Trial 02: flipped K bit #113, Bits different = 124 (48.44%)
  Trial 03: flipped K bit #126, Bits different = 150 (58.59%)
  Trial 04: flipped K bit #53, Bits different = 127 (49.61%)
  Trial 05: flipped K bit #18, Bits different = 132 (51.56%)
  Trial 06: flipped K bit #65, Bits different = 132 (51.56%)
  Trial 07: flipped K bit #121, Bits different = 128 (50.00%)
  Trial 08: flipped K bit #99, Bits different = 116 (45.31%)
  Trial 09: flipped K bit #72, Bits different = 123 (48.05%)
  Trial 10: flipped K bit #82, Bits different = 131 (51.17%)
Average bits different (B): 129.50 / 256 (50.59%)
```

```
Summary (B) Key bit flip:
Trial | Flipped bit idx (K) | Bits different | %
------+---------------------+----------------+-------
01    | 20                  | 132            | 51.56%
02    | 113                 | 124            | 48.44%
03    | 126                 | 150            | 58.59%
04    | 53                  | 127            | 49.61%
05    | 18                  | 132            | 51.56%
06    | 65                  | 132            | 51.56%
07    | 121                 | 128            | 50.00%
08    | 99                  | 116            | 45.31%
09    | 72                  | 123            | 48.05%
10    | 82                  | 131            | 51.17%
```

Average Bits Different: 129.50 / 256 = 0.505859

Average % Different: 50.59%

In this test, we flipped a single bit in the encryption key and measured how much the ciphertext changed. Over 10 trials, the average change was about 129.5 out of 256 bits (50.59%), which is almost exactly half of the ciphertext bits. This confirms that even a tiny change in the key causes a widespread and unpredictable change in the output, demonstrating AES's strong avalanche property with respect to key variations.

CBC Mode Extended Analysis

a) Bit Error in Ciphertext

```
Extended Analysis (CBC)

CBC: Single-bit error in ciphertext
Original plaintext blocks (3 blocks):
  [01] 09032ab3ce193b941e2fd500308333e5
  [02] b87865a3df8109e6aa471f99b253b8f1
  [03] d5803b80f38f8ff7d3577d50673e4596
Original ciphertext blocks (4 blocks):
  [01] 249b922168bbaa06e0c66922dcc8cfd2
  [02] d4834d310d0a0d909e635571d92d938e
  [03] 399587120ee3d97059222ef47a678999
  [04] 7876f8113093b3d82bb9a73d5b8d7267

Flipped ciphertext bit #137 (in ciphertext block index 1)

Decrypted plaintext blocks after bit flip (3 blocks):
  [01] 09032ab3ce193b941e2fd500308333e5
  [02] 7a6c1ef7875436132b56858fcc5d2e6d
  [03] d5823b80f38f8ff7d3577d50673e4596

Analysis for CBC single-bit flip:
  Affected plaintext blocks: 1, 2
  - Block 1: heavily corrupted (62 bits differ out of 128).
  - Block 2: ~1-bit error observed (1 bit(s) differ).
```

• **Which blocks are affected in the decrypted message?**
Plaintext blocks [01] and [02] are affected; [03] is unaffected. Block [01] is heavily corrupted (62/128 bits differ), and block [02] shows about a 1-bit error.

• **How many plaintext blocks are corrupted as a result?**
Two blocks show changes: the first block is heavily corrupted and the second has a single-bit error.

• **Why does this behavior occur in CBC mode?**
In CBC mode, flipping a single bit in a ciphertext block C1 causes the corresponding plaintext block P1 to become heavily scrambled because decrypting the altered block changes most of its bits due to the block cipher's avalanche effect, and it also causes exactly one bit error in the next plaintext block P2 since that block's decryption is correct but is XORed with the modified C1; blocks after P2 remain unaffected because their ciphertext chain is intact.

b) Loss of a Ciphertext Block

```
CBC: Loss of a full ciphertext block
Original ciphertext blocks (5 blocks):
  [01] 2e64f093b9b6378bd75e2f253c4f3c62
  [02] 4d31cf5acb07b046a8c5ada25790a9da
  [03] 348213e4c8403ef3d74e8111166fac6a
  [04] 1994fb827f4f498e3ed0ca3082b774d8
  [05] c837aa79f49063e5fc1080159459794a

Simulated loss at ciphertext block index 1

Decrypted plaintext blocks (5 blocks):
  [01] 442cd7f92e7c167ca5b5f594dd992040
  [02] 00000000000000000000000000000000
  [03] d8518f46f579de8edffae17623a09c01
  [04] b5ad5c10189588ee7a4432964f6b96b7
  [05] 10101010101010101010101010101010
```

- **Which blocks are affected?**

The lost ciphertext block [02] is unrecoverable, and the next plaintext block [03] decrypts wrong because it needed the missing [02] for the XOR.

- **Can any block still be decrypted correctly?**

Yes. [01] decrypts fine (it doesn't depend on the missing block), and from [04] onward the chain recovered, so [04] and [05] decrypt correctly.

- **What does this reveal about error propagation in CBC?**

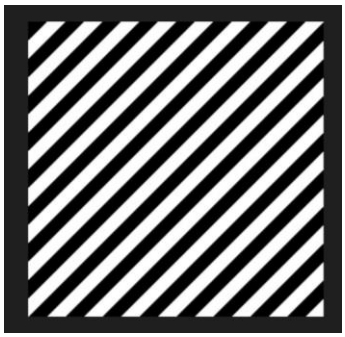
Losing one whole ciphertext block in CBC destroys two plaintext blocks (the lost one and its following block); after that, decryption recovers and proceeds correctly.

c) Data Exposure in Ciphertext (Image Test)

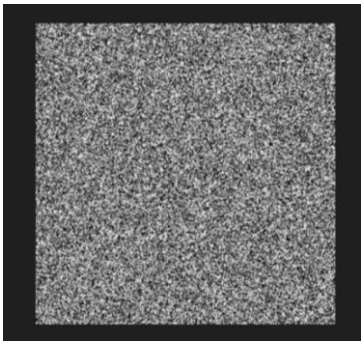
We encrypted a generated 256x256 image with AES-128 CBC.

```
----------------------------------------------------------------------
CBC: Data Exposure image
Generated 256x256 black/white checkerboard as our image before.
  out\cbc_image_demo_plain.png
  out\cbc_image_demo_cipher_visual.png
Done.
PS C:\Users\user\OneDrive\Desktop\crypto_task2>
```

Plain image: (out/cbc_image_demo_plain.png)



Ciphertext visual: (out/cbc_image_demo_cipher_visual.png)



- **What does the reconstructed image look like?** The reconstructed image appears completely unclear and noisy, with no clear shapes or recognizable features.

- **Can any recognizable patterns be seen?** No, there are no visible patterns, repeated areas, or color structures from the original image.

- **What does this imply about CBC mode's resistance to data leakage compared to ECB?** This shows that CBC effectively hides information. Unlike ECB, where repeated patterns in plaintext can appear in the ciphertext, CBC mixes each block with the previous one before encryption. This ensures that even identical blocks produce different ciphertext, keeping the image’s structure and patterns completely confidential.